



Concetti Base: tensor, device, autograd

NumPy vs PyTorch

Cosa condividono e cosa li distingue

Tensor Libraries

Entrambe sono librerie Python per gestire array n-dimensionali (Vettori, Matrici, Tensori).

PyTorch ha due funzionalità in più molto utili:

1. **Devices:** gli array possono essere elaborati da dispositivi diversi (CPU, GPU, NPU, ...).
2. **Autograd:** meccanismo integrato e automatico per il calcolo del gradiente.

numpy.ndarray
<i>data</i>
<i>shape : tuple</i>
<i>dtype : numpy.dtypes</i>

False: Autograd disabilitato
True: Autograd abilitato

Riferimento all'operazione che ha calcolato il
tensore, al fine di poter calcolare il gradiente

torch.Tensor
<i>data</i>
<i>shape: torch.Size</i>
<i>dtype: torch.dtype</i>
<i>device: torch.device</i>
<i>requires_grad: bool</i>
<i>grad_fn: None function</i>
<i>grad: None torch.Tensor</i>

Identifica il device
responsabile del
tensore

conterrà il
gradiente a valle
del calcolo con
Autograd

Codice a confronto

Le stesse operazioni scritte in NumPy e in PyTorch (La maggioranza delle funzioni sono identiche)

NumPy

```
import numpy as np

# Creazione
# default: dtype=np.float64
a = np.array([[1., 2.], [3., 4.]])
b = np.ones((2, 2))
c = np.random.randn(128, 128, 3)

# Conversione
d = np.float16(b)

# Operazioni
e = (a + b) ** 2
f = np.stack((a, e), 0)
g = np.reshape(c, (64, 256, 3))
v = np.transpose(c, (2,1,0))
m = np.mean(f)
```

PyTorch

```
import torch

# Creazione
# default: dtype=torch.float32, device='cpu'
a = torch.tensor([[1., 2.], [3., 4.]])
b = torch.ones((2, 2))
c = torch.randn(128, 128, 3)

# Conversione
d = b.to(torch.float16)

# Operazioni
e = (a + b) ** 2
f = torch.stack((a, e), 0)
g = torch.reshape(c, (64, 128, 3))
v = torch.permute(c, (2,0,1))
m = torch.mean(f)
```

Interoperabilità

```
t = torch.from_numpy(a) # from numpy array to pytorch tensor
a = t.cpu().numpy()     # from pytorch tensor to numpy array
```

Devices in PyTorch

Ogni tensore è gestito da uno specifico device

Device rules

1. Ogni *device* è identificato da un stringa univoca ('cpu', 'cuda:0', 'xpu', ...)
2. Ogni tensore è gestito da uno specifico *device*
3. Non si possono fare elaborazioni tra tensori gestiti da *device* diversi
4. Un tensore può essere spostato esplicitamente da un *device* ad un altro

```
# 1. Creazione tensore associando il device
a = torch.zeros((3, 4), device='cpu') # di CPU
b = torch.ones((4, 2), device='cuda:0') # di GPU
c = torch.randn(4, 2, device=b.device) # di GPU

# 2. Operazioni tra tensori
d = b + c # il risultato è un tensore della GPU
e = a @ c # ERRORE, operazione non consentita

# 3. Spostamento Esplicito
f = a.to('cuda:0') # da CPU a GPU
e = f @ c # operazione consentita tra tensori di GPU
g = f.cpu() # da GPU a CPU
h = b.to(g.device) # da GPU a CPU
```

Device types

- **'cpu'**: identifica il processore centrale del computer (default).
- **'cuda:0', 'cuda:1', ...**: identifica le GPU NVIDIA basate su tecnologia CUDA.
- **'mps'**: identifica il chip Apple Silicon (Metal Performance Shaders).
- **'xpu:0', 'xpu:1', ...**: identifica le GPU Intel basate su backend XPU.

Autograd — Differenziazione Automatica

Il calcolo del gradiente è fondamentale per l'addestramento delle reti neurali

$$z(x, y) = x * (y^2 + 2)$$

$z(3.5, 2.0)$?

1. Tensori di partenza

```
x = torch.tensor(3.5)
```

```
y = torch.tensor(2.0)
```

2. Esegui la funzione tramite operazioni tra tensori

```
w = y**2 + 2
```

```
z = x * w    # z(3.5, 2.0) = 21
```

Autograd — Differenziazione Automatica

Il calcolo del gradiente è fondamentale per l'addestramento delle reti neurali

$$z(x, y) = x * (y^2 + 2)$$

$$z(3.5, 2.0) = 21$$

$$\frac{\partial z}{\partial x}(3.5, 2.0) ?$$

$$\frac{\partial z}{\partial y}(3.5, 2.0) ?$$

1. Tensori di partenza con calcolo del gradiente attivo

```
x = torch.tensor(3.5, requires_grad=True)
```

```
y = torch.tensor(2.0, requires_grad=True)
```

2. Esegui la funzione tramite operazioni tra tensori

```
w = y**2 + 2
```

```
z = x * w    # z(3.5, 2.0) = 21
```

1

requires_grad=True

Attiva Autograd

Impostare a True il *requires_grad* dei tensori di partenza.

2

Forward pass

Esegui operazioni

Ogni operazione sui tensori viene registrata costruendo il grafico delle relazioni funzionali dinamicamente.

Autograd — Differenziazione Automatica

Il calcolo del gradiente è fondamentale per l'addestramento delle reti neurali

$$z(x, y) = x * (y^2 + 2)$$

$$z(3.5, 2.0) = 21$$

$$\frac{\partial z}{\partial x}(3.5, 2.0) ?$$

$$\frac{\partial z}{\partial y}(3.5, 2.0) ?$$

```
# 1. Tensori di partenza con calcolo del gradiente attivo
x = torch.tensor(3.5, requires_grad=True)
y = torch.tensor(2.0, requires_grad=True)

# 2. Esegui la funzione tramite operazioni tra tensori
w = y**2 + 2
z = x * w    # z(3.5, 2.0) = 21

# 3. Calcolo dei gradienti (Backward)
z.backward()
```

1

requires_grad=True

Attiva Autograd

Impostare a True il `requires_grad` dei tensori di partenza.

2

Forward pass

Esegui operazioni

Ogni operazione sui tensori viene registrata costruendo il grafico delle relazioni funzionali dinamicamente.

3

.backward()

Calcolo dei gradienti

Percorre automaticamente il grafo al contrario e calcola il gradiente per ogni tensore di partenza.

Autograd — Differenziazione Automatica

Il calcolo del gradiente è fondamentale per l'addestramento delle reti neurali

$$z(x, y) = x * (y^2 + 2)$$

$$z(3.5, 2.0) = 21$$

$$\frac{\partial z}{\partial x}(3.5, 2.0) = 6$$

$$\frac{\partial z}{\partial y}(3.5, 2.0) = 14$$

1. Tensori di partenza con calcolo del gradiente attivo

```
x = torch.tensor(3.5, requires_grad=True)
```

```
y = torch.tensor(2.0, requires_grad=True)
```

2. Esegui la funzione tramite operazioni tra tensori

```
w = y**2 + 2
```

```
z = x * w # z(3.5, 2.0) = 21
```

3. Calcolo dei gradienti (Backward)

```
z.backward()
```

3. Leggi i gradienti

```
x.grad # dz/dx(3.5, 2.0) = 6
```

```
y.grad # dz/dy(3.5, 2.0) = 14
```

1

requires_grad=True

Attiva Autograd

Impostare a True il `requires_grad` dei tensori di partenza.

2

Forward pass

Esegui operazioni

Ogni operazione sui tensori viene registrata costruendo il grafico delle relazioni funzionali dinamicamente.

3

.backward()

Calcolo dei gradienti

Percorre automaticamente il grafo al contrario e calcola il gradiente per ogni tensore di partenza.

4

.grad

Leggi i gradienti

Il gradiente rispetto ad ogni tensore di partenza è conservato nel relativo `grad`



Ecosistema PyTorch

Le librerie ufficiali PyTorch

torch.nn

Neural Network Modules

Layers (Linear, Conv2d, ...), loss functions (MSELoss, CrossEntropyLoss...). I mattoni fondamentali per costruire architetture di reti neurali.

```
model = torch.nn.Sequential(
    torch.nn.Linear(784, 256),
    torch.nn.ReLU(),
)
```

torch.optim

Optimization Algorithms

SGD, Adam, AdamW, RMSProp... Gestiscono automaticamente l'aggiornamento dei pesi e l'azzeramento dei gradienti.

```
opt = torch.optim.SGD(
    model.parameters(),
    lr=1e-3
)
```

torch.utils.data

Data Management

Utilità per gestire grandi quantità di dati durante training e testing.

```
loader = torch.utils.data.DataLoader(
    dataset, batch_size=2,
    shuffle=True
)
```

torchvision

Computer Vision

Utilità per immagini, dataset di immagini (ImageNet, CIFAR...), e operazioni di augmentation.

```
from torchvision.models import resnet50
model = resnet50(pretrained=True)
```

torchaudio

Audio & Speech

Utilità for audio, dataset di audio (LibriSpeech, VoxCeleb), e modelli (wav2vec2, HuBERT) per speech processing.

```
waveform, sr = torchaudio.load('audio.wav')
```

torchtext / torchtext

NLP & Text

Natural Language Processing (NLP), Tokenizzazione, LLM.

```
from torchtext import vocab
vocab.build_vocab_from_iterator(tokens)
```

<https://pytorch.org/domains/>



Ecosistema HuggingFace

Modelli pre-addestrati, tokenizzatori, diffusione e hub

huggingface_hub

Hugging Face Hub è una piattaforma con oltre 2M di modelli neurali pre-addestrati e 500k dataset.

timm

PyTorch Image Models: libreria per architetture per l'elaborazione di immagini (DINO, ViT, ConvNeXt, Swin Transformer). Fine-tuning semplificato.

transformers

La principale libreria per i modelli Transformer pre-addestrati (GPT-2, LLaMA, ViT) for testo, immagini ed audio.

tokenizers

Tokenizzazione ultra-veloce. Componente fondamentale per preparare il testo in input ai transformer.

diffusers

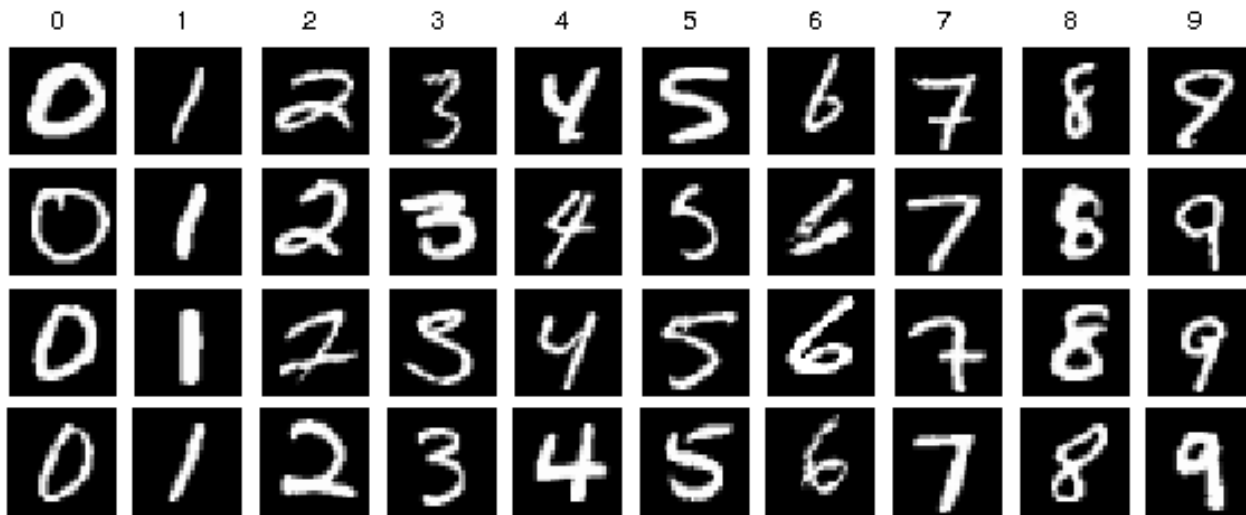
Modelli di diffusione per generazione di immagini, audio e video. Include Stable Diffusion, FLUX, scheduler e pipeline modulari.

Classificazione con LeNet

Concetti Base: Convolution layer, Attivazione, Pooling, Softmax

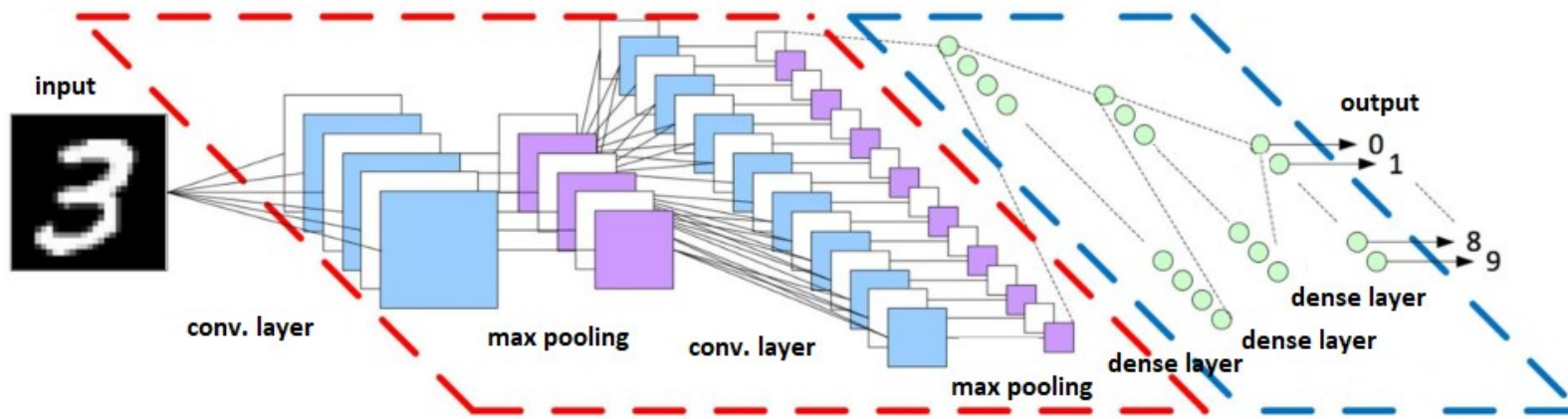
MNIST handwritten digits recognition

- Il MNIST database contiene immagini (28 x 28 pixel) di cifre scritte a mano, 60.000 immagini di training e 10.000 di test.
- L'obiettivo è quello di riconoscere la cifra scritta, associando ogni immagine ad una delle 10 possibili classi (0, 1, 2, ... , 9)



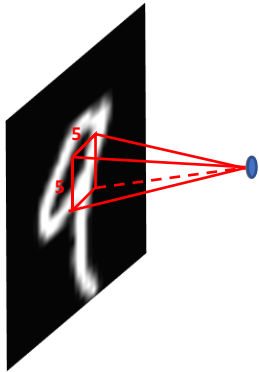
Architettura LeNet

- Consiste di due parti:
 - un blocco di strati convoluzionali (per l'estrazione delle feature)
 - un blocco di strati fully connected (per la classificazione)

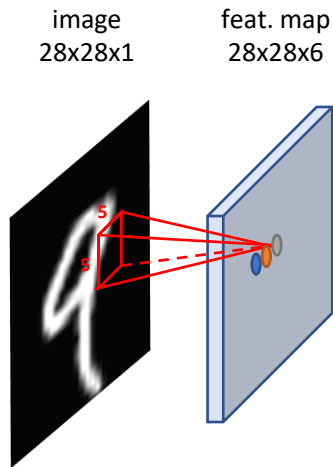


Architettura LeNet

image
28x28x1



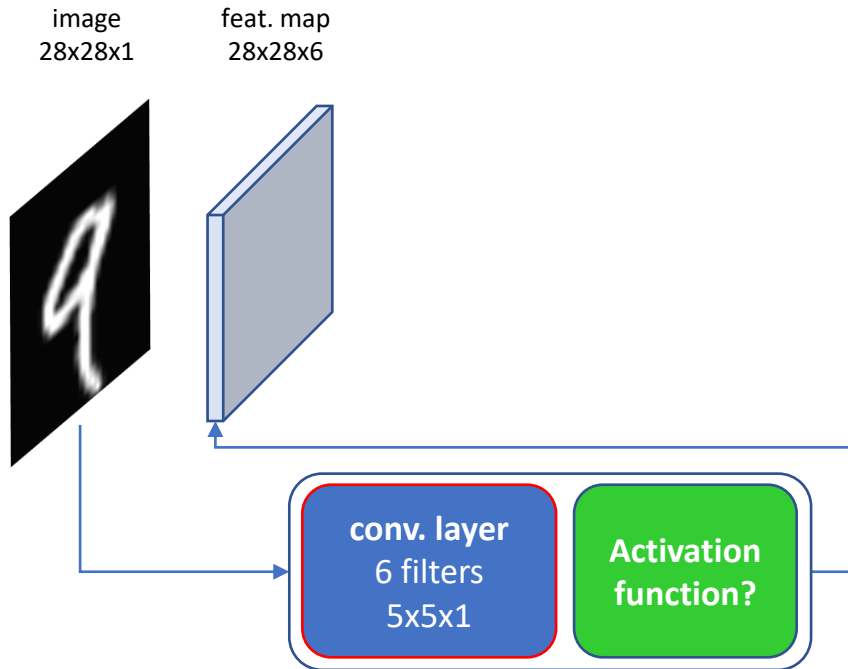
Architettura LeNet



$$F(i, j, c) = \underbrace{b_c}_{6} + \sum_m \sum_n \sum_k I(i + m, j + n, k) \underbrace{w_c(m, n, k)}_{5 \times 5 \times 1 \times 6} \quad c = 0, \dots, 5$$

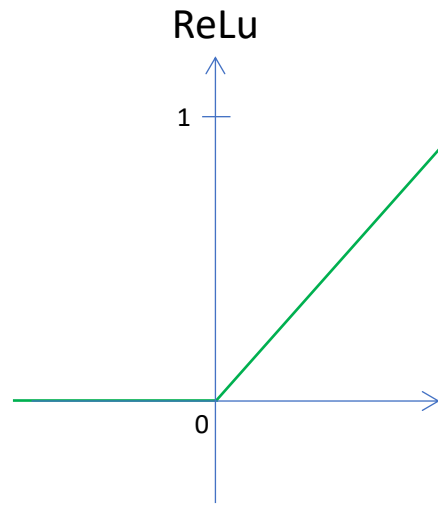
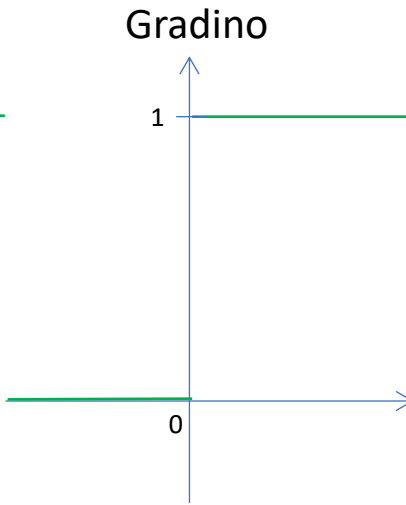
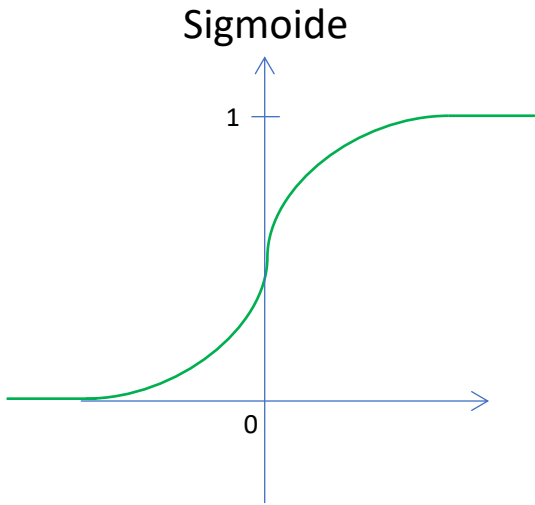
156 parameters

Architettura LeNet



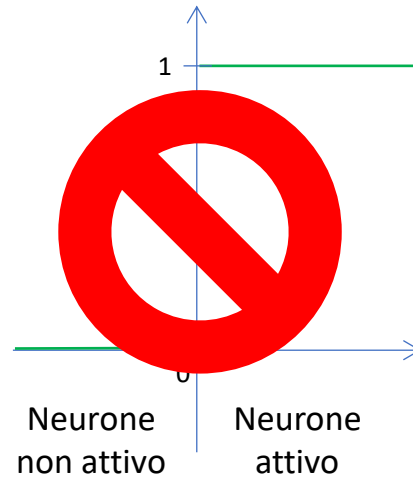
Funzione di attivazione

- Quale usare?



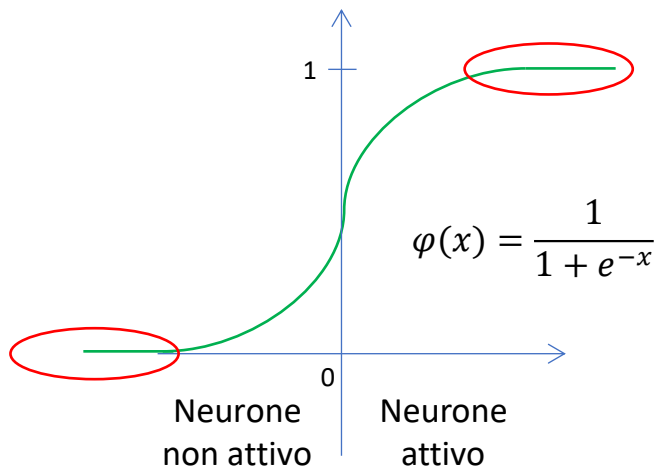
Funzione di attivazione: Gradino

- La funzione di attivazione adottata nella definizione del perceptrone.
- La sua derivata è sempre nulla, non è si può applicare la discesa del gradiente



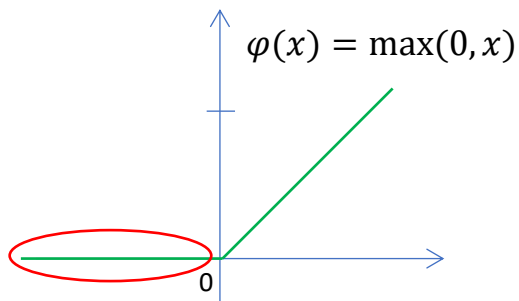
Funzione di attivazione: Sigmoide

- Proposta in LeNet per sostituire il gradino
- La sua derivata non è sempre nulla, si può utilizzare la discesa del gradiente
- La zona di saturazione (in rosso) facilitano la possibilità di un vanishing gradient

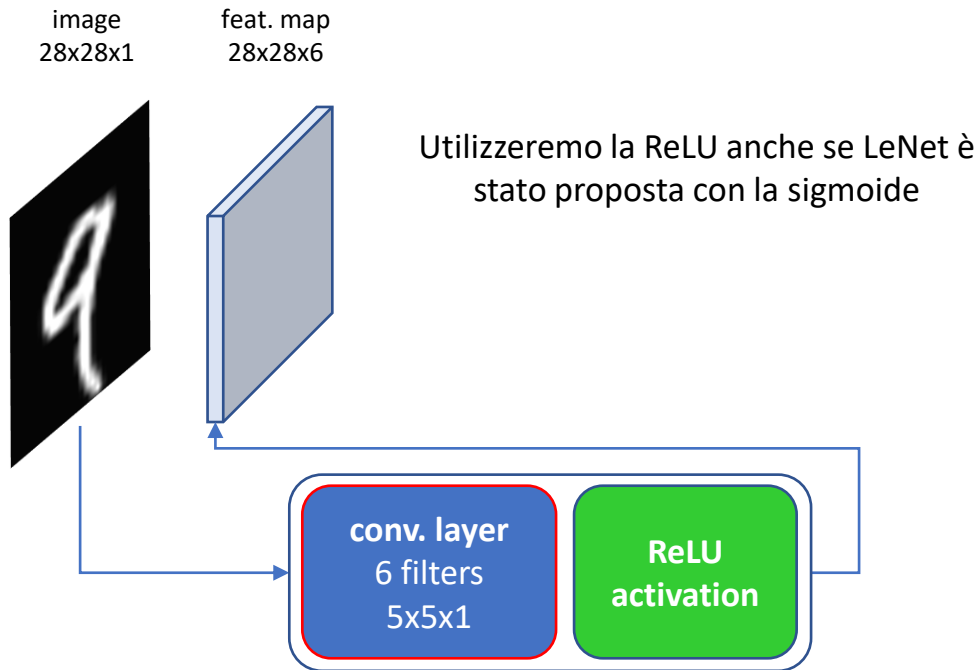


Funzione di attivazione: ReLU (Rectified Linear Unit)

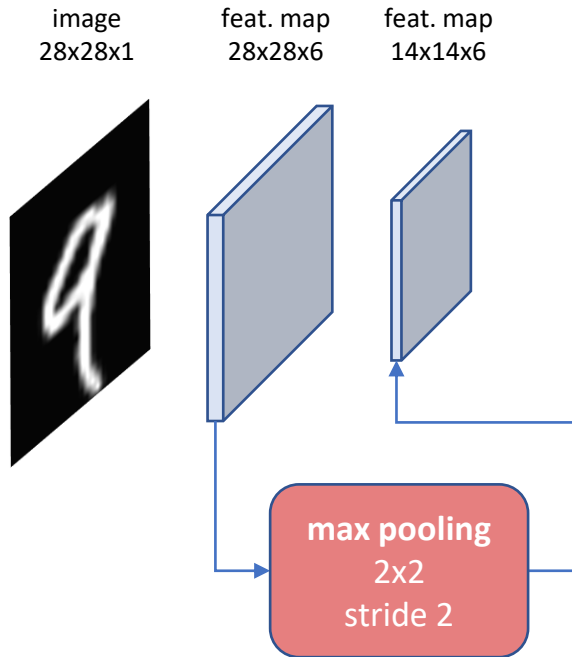
- Riduce la possibilità del vanishing gradient.
- Sperimentalmente è stata dimostrata un convergenza dell'addestramento circa 6 volte più veloce rispetto all'utilizzo della sigmoide.
- Ulteriori varianti proposte in letteratura puntano ad diminuire ulteriormente la possibilità che capiti il vanishing gradient.



Architettura LeNet



Architettura LeNet



Max pooling

- Le operazioni di pooling riducono le dimensioni spaziali per ridurre la complessità degli strati successive.
- A blocchi disgiunti si applica una operazione fissata (es. max pooling, median pooling, average pooling)

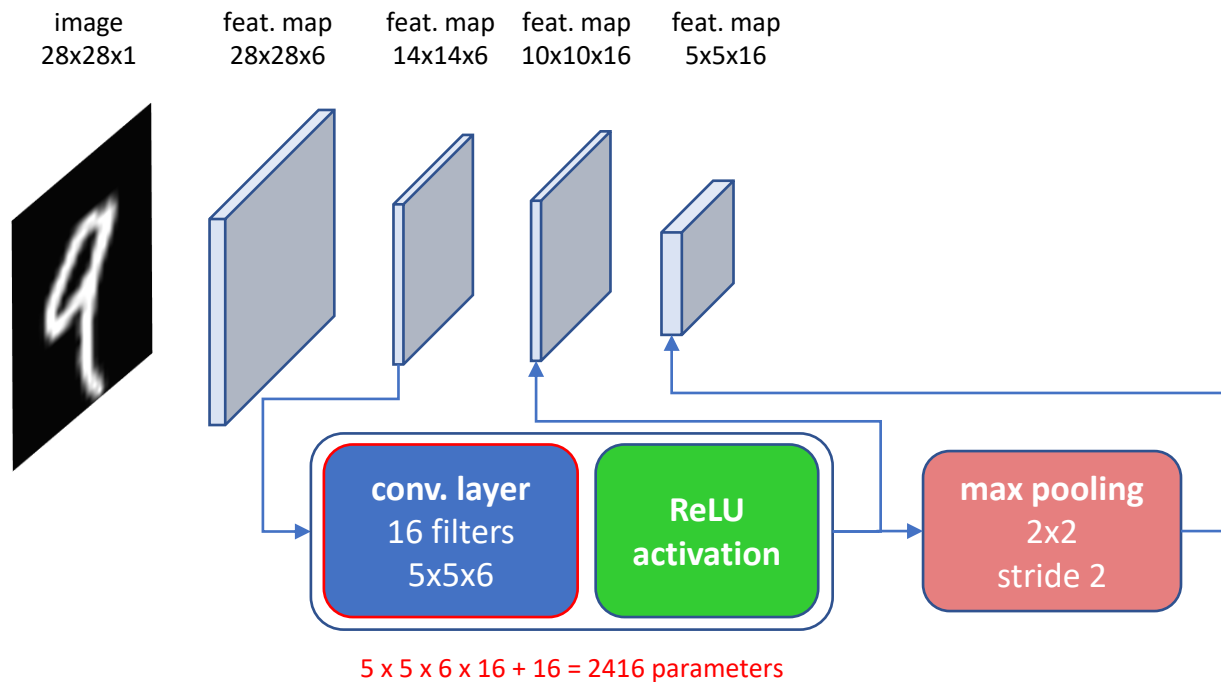
before pooling

12	20	30	0
8	12	2	0
34	70	37	4
112	100	25	12

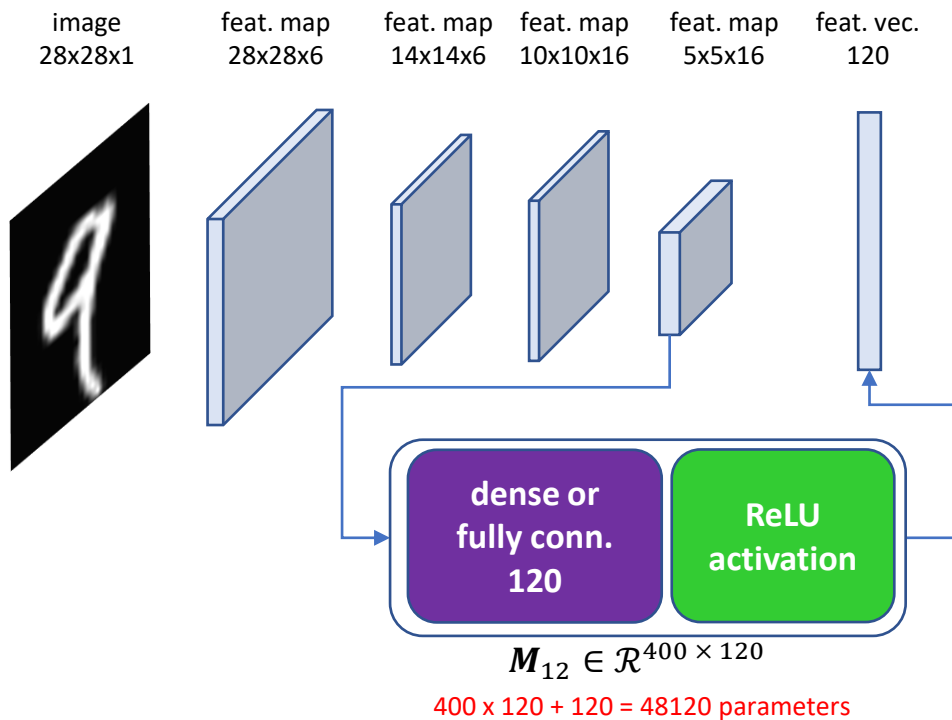
after max pooling
2x2, stride 2

20	30
112	37

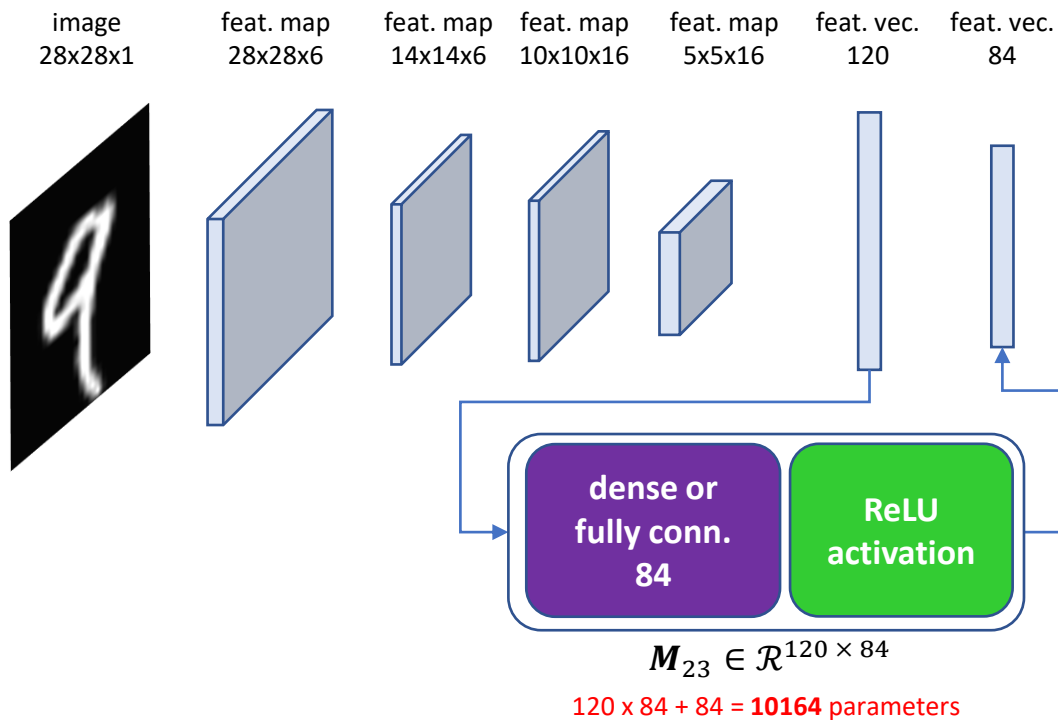
Architettura LeNet



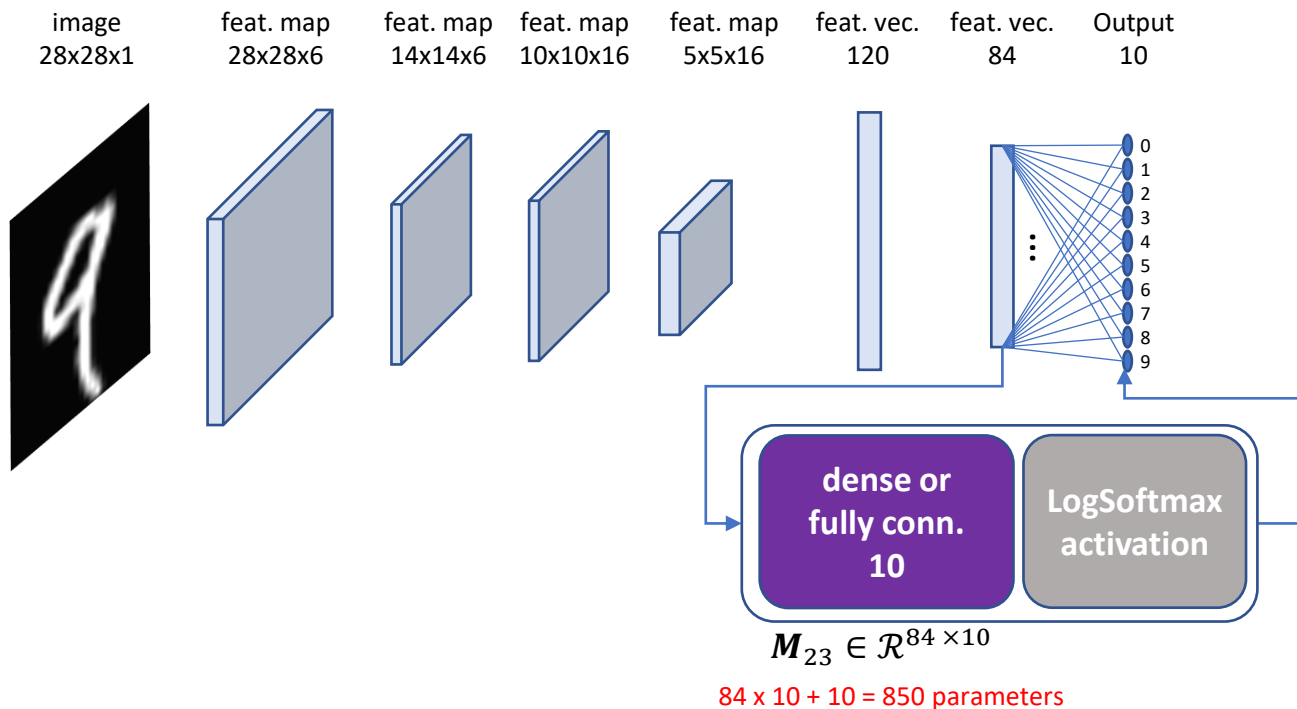
Architettura LeNet



Architettura LeNet



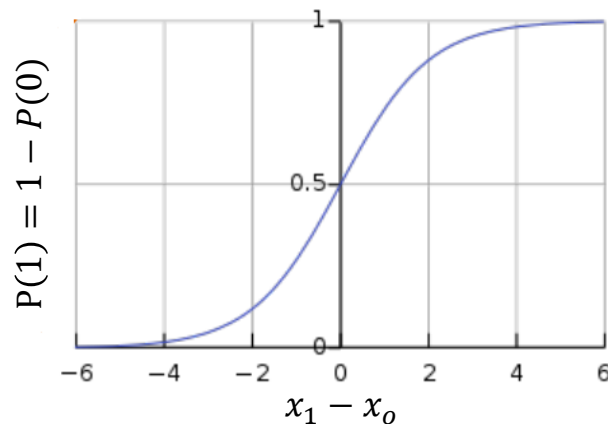
Architettura LeNet



Softmax

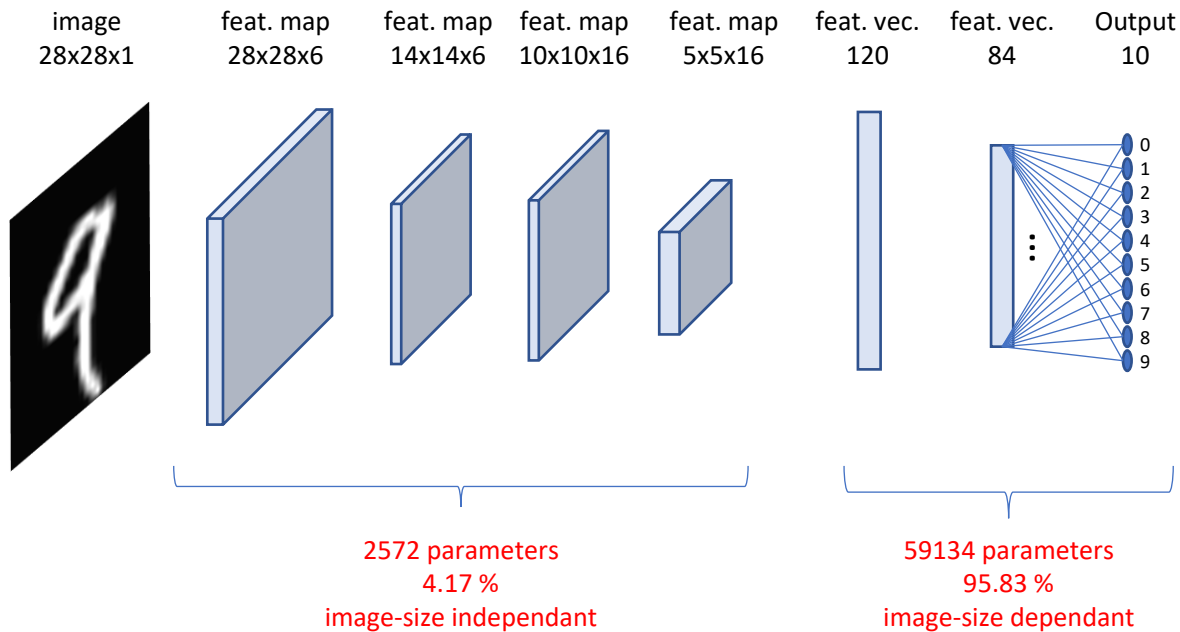
- La funzione Softmax garantisce che i valori siano positivi e a somma unitaria
- I valori ottenuti si possono interpretare come probabilità delle classi

$$P(i) = \frac{e^{x_i}}{\sum_i e^{x_i}}$$



Possiamo disegnare
la Softmax in caso di
2 classi

Architettura LeNet



- Colab è un servizio gratuito fornito da Google, basato su Jupyter Notebook che permette di eseguire codice python sui server google. Per utilizzarlo è necessario solo un Google account (gmail.com). Colab va attivato alla pagina web: <https://gsuite.google.com/u/1/marketplace/app/colaboratory/1014160490159>
- Si può lavorare online e il codice è salvato in Google Drive.
- Per creare un nuovo Colab Notebook, vai sulla pagina di Google Drive e clicca “New”, scegli “More” e seleziona “Colaboratory”.
- Per scegliere la versione di Python e l’uso della GPU, clicca “Notebook settings” nel menù “edit”.

Passi dell'esercitazione

1. Preparazione dei dati MNIST;
2. Definizione dell'architettura LeNet;
3. Addestramento;
4. Analisi delle prestazioni.

(1) Preparare Training, Validation, e Test set

- Training set: I dati usati per addestrare la rete
- Validation set: I dati usati per settare i parametri ed evitare overfitting
- Test set: I dati usati per testare le prestazioni

- Suddivisione per MNIST
 - Training set : 55000 immagini
 - Validation set : 5000 immagini
 - Test set : 10000 immagini

Training (78,7%)

Validation (7%)

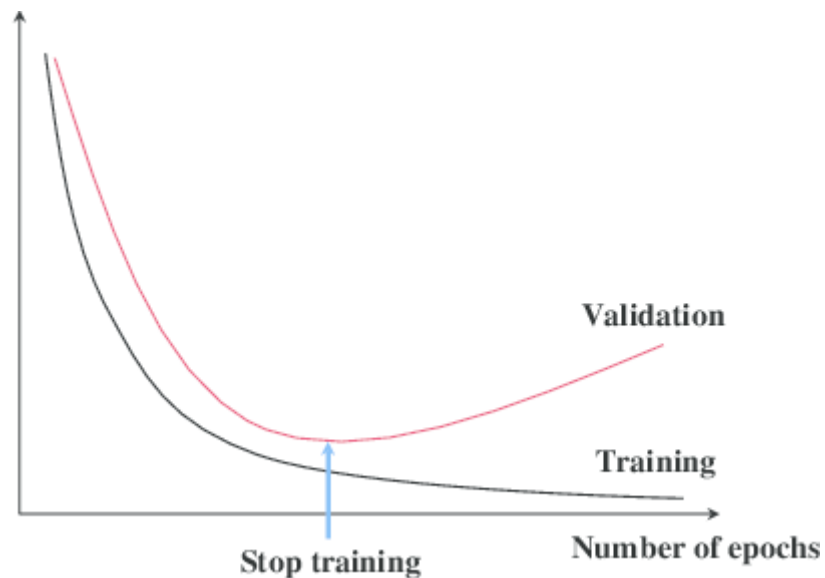
Testing (14,3%)

(2) Definire l'architettura LeNet

```
Sequential(  
  (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))  
  (1): ReLU()  
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))  
  (4): ReLU()  
  (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  (6): Flatten(start_dim=1, end_dim=-1)  
  (7): Linear(in_features=400, out_features=120, bias=True)  
  (8): ReLU()  
  (9): Linear(in_features=120, out_features=84, bias=True)  
  (10): ReLU()  
  (11): Linear(in_features=84, out_features=10, bias=True)  
  (12): LogSoftmax(dim=None)  
)
```

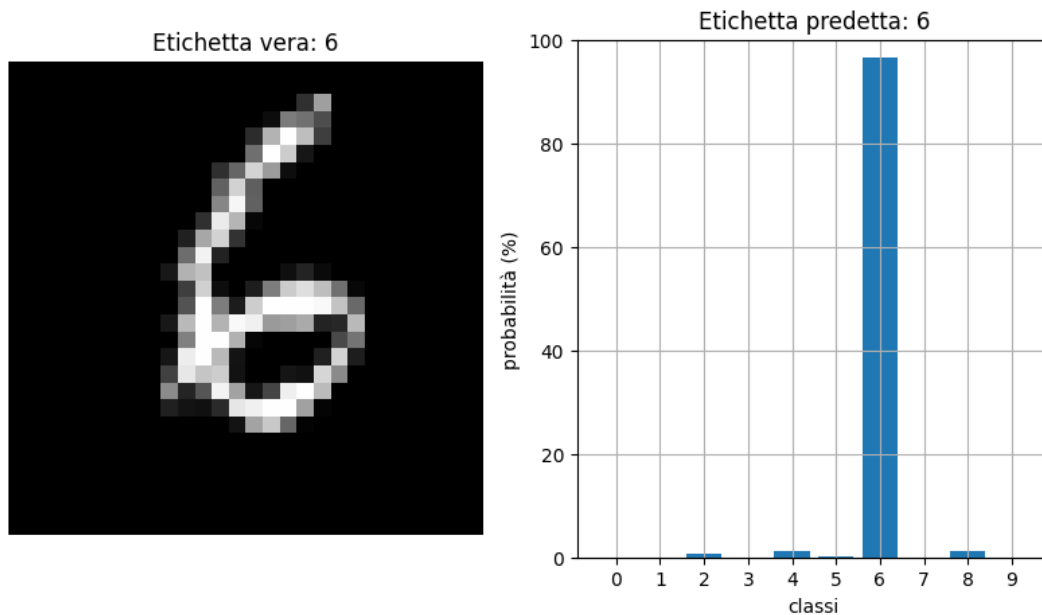
(3) Addestrare

- Applicare l'algoritmo di Stochastic Gradient Descent per ottimizzare i parametri della rete sul set di addestramento.
- Monitorare le prestazioni sul set di validazione per evitare l'overfitting.



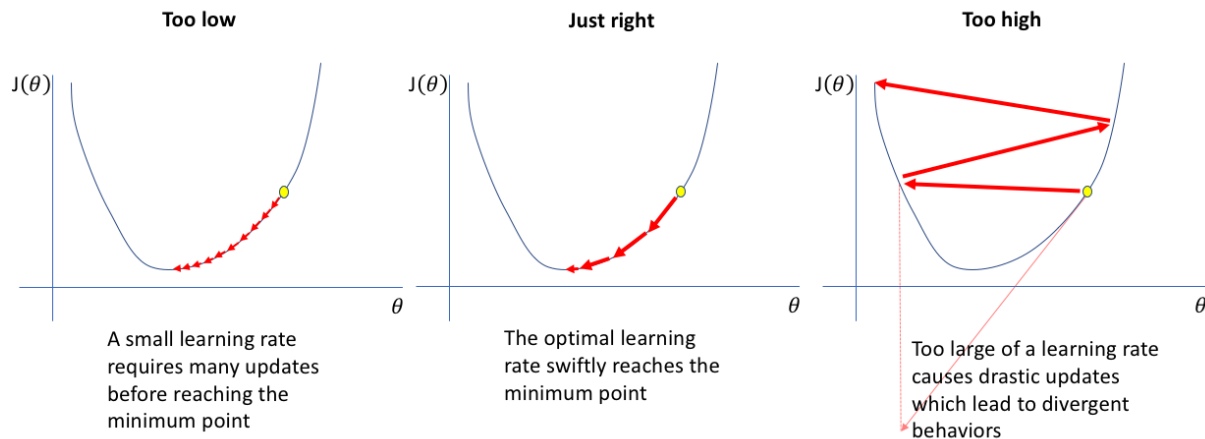
(4) Valutare le prestazioni sul test set

- Valutare le prestazioni sui dati non presenti nel training



Come settare il learning rate

- Se la loss function decresce lentamente, bisogna aumentare il learning rate
- Se la loss function non decresce, diminuisci il learning rate
- Se la loss function oscilla, diminuisci il learning rate



CIFAR-10 dataset

- Consiste di 60.000 32x32 immagini a colori su 10 classi, con 6000 immagini per classe. Ci sono 50000 immagini di training e 10.000 di test

